

Webプログラミング レポート課題

GitHub リンク

https://github.com/Kuro-aletta/webpro_06

学生番号 25G1020

氏 名 薄井 悠希

目次

1	はじめに	1
1.1	開発の目的	1
1.2	作成したシステム	1
2	利用者向けドキュメント	1
2.1	概要	1
2.2	起動画面（メインメニュー）	2
2.3	一覧表示	3
2.4	詳細表示	3
2.5	データの編集	4
2.6	データの追加	7
2.7	データの削除	10
3	管理者向けドキュメント	12
3.1	インストール方法	12
3.2	起動方法	13
3.3	起動できない場合	13
3.4	終了方法	14
3.5	分かっている不具合・制限事項	14
4	開発者向けドキュメント	15
4.1	概要	15
4.2	開発環境	15
4.3	ディレクトリ構造	15
4.4	システム構造と URL 設計	16
4.5	データ構造	19
4.6	各システムの機能詳細	20
4.7	講義範囲外の要素の実装	21
5	おわりに	22

1 はじめに

1.1 開発の目的

本レポートでは、Web アプリケーション開発の基礎技術習得を目的として作成したシステムについて報告する。近年の Web システムにおいて不可欠な技術である、サーバサイドでの動的なコンテンツ生成を実現する Node.js および Express フレームワークを利用し、データの「新規登録 (Create)」「一覧, 詳細表示 (Read)」「編集・更新 (Update)」「削除 (Delete)」という基本機能 (CRUD) を備えたアプリケーションを構築した。本開発では、操作性や画面遷移を統一した 3 つの異なるシステムを作成することで、データ構造が異なっても共通の設計思想で実装可能であることを確認した。

1.2 作成したシステム

本課題では、以下の 3 つのシステムを作成した。

1. **成績管理システム**：履修した科目の単位数や成績を記録し、学修状況を管理するシステム。
2. **教員情報システム**：所属学科や職位、担当科目などの教員情報を一元管理し、閲覧するシステム。
3. **お小遣い帳システム**：日々の支出を記録し、合計金額を自動計算して表示する金銭管理システム。

2 利用者向けドキュメント

本節では、作成したシステムのうち「お小遣い帳システム」を例に、利用方法について説明する。なお、他の 2 つのシステムについても操作方法は統一されており、同様の手順で利用可能である。

2.1 概要

本システムを利用するためには、Google Chrome や Microsoft Edge などの Web ブラウザが必要である。また、システムを利用する前に、管理者によって Web サーバが起動されている必要がある。お小遣い帳システムでは、日々の買い物の「日付」「項目」「値段」などを記録し、一覧画面で支出の合計を確認することができる。

2.2 起動画面（メインメニュー）

Web ブラウザのアドレスバー（図 1）に `http://localhost:8080/` と入力してアクセスすると、メインメニューが表示される（図 2）。ここから「お小遣い帳」のリンクをクリックすることで、お小遣い帳システムへ移動できる。万が一、存在しないページへアクセスした場合は「ページが見つかりません」と表示され、トップページへ戻るリンクが提供される（図 3）。



図 1: Google Chrome の場合のアドレスバーの位置

メインメニュー

- [成績管理システム](#)
- [教員情報システム](#)
- [お小遣い帳](#)

図 2: メインメニュー画面

ページが見つかりません

[トップページへ](#)

図 3: 存在しないページへアクセスした場合の画面

2.3 一覧表示

お小遣い帳システムにアクセスすると、現在登録されているデータの一覧が表示される(図4)。表には「日付」「項目」「値段」が表示され、表の最下部には登録されている全データの合計金額が自動計算されて表示される。各項目のリンクをクリックすると、そのデータの詳細画面へ遷移する。

お小遣い帳

[新規項目を追加する](#)

日付	項目名	値段
2025-12-10	カレー	250 円
2025-12-12	おーいお茶	120 円
2025-12-15	ゲーム課金	30000 円
合計金額		30370 円

← 各項目の値段を全て足した金額
 $250 + 120 + 30000$

[トップページへ](#)

図 4: お小遣い帳の一覧表示画面（合計金額が表示される）

2.4 詳細表示

一覧画面で項目名をクリックすると、詳細表示画面へ遷移する(図5)。ここでは、一覧には表示しきれなかった「購入目的」や「備考欄」を含むすべての情報を確認できる。また、この画面からデータの「編集」や「削除」を行うためのリンクが配置されている。

お小遣い帳

[新規項目を追加する](#)

日付	項目名	値段
2025-12-10	カレー	250 円
2025-12-12	おーいお茶	120 円
2025-12-15	ゲーム課金	30000 円
合計金額		30370 円

[トップページへ](#)

ゲーム課金

項目	データ
値段	30000 円
日付	2025-12-15
購入目的	ガチャ
備考欄	100連爆死 /(^o^)\

[編集する](#)
[一覧に戻る](#)

[この項目を削除する](#)

図 5: 詳細表示の内容と仕方

2.5 データの編集

登録済みのデータを修正する手順は以下の通りである。

1. 修正したいデータの詳細画面を開き、「編集する」リンクをクリックする（図6）。

お小遣い帳

[新規項目を追加する](#)

日付	項目名	値段
2025-12-10	カレー	250 円
2025-12-12	おーいお茶	120 円
2025-12-15	ゲーム課金	30000 円
合計金額		30370 円

[トップページへ](#)

ゲーム課金

項目	データ
値段	30000 円
日付	2025-12-15
購入目的	ガチャ
備考欄	100連爆死 /(^o^)\

[編集する](#)
[一覧に戻る](#)

[この項目を削除する](#)

図 6: 編集画面への行き方

2. 編集画面が表示される（図7）。現在のデータが入力欄に入った状態で表示されるため、修正したい箇所のみ書き換える。

ゲーム課金の編集

項目	データ入力欄
日付	2025/12/15
項目	ゲーム課金
値段	30000 円 ※半角数字で入力
購入目的	ガチャ
備考欄	100連爆死 /(^o^)\

確認画面へ

[キャンセル（詳細に戻る）](#)

図 7: 編集画面

3. 修正が完了したら「確認画面へ」ボタンを押す（図8）。

ゲーム課金の編集

項目	データ入力欄
日付	2025/12/15
項目	ゲーム課金
値段	60000 円 ※半角数字で入力
購入目的	ガチャ
備考欄	200連UR3体 (*°▽°*)

確認画面へ

[キャンセル（詳細に戻る）](#)

編集例

日付: (変更なし)

項目名: (変更なし)

値段: **30000円 → 60000円**

購入目的: (変更なし)

備考欄: **100連爆死 /(^o^)**
→ **200連UR3体 (*°▽°*)**

図 8: 編集例と「確認画面へ」ボタン

4. 表示された確認画面で変更内容を確認する（図9）。修正が必要な場合は「修正する」ボタンを押して編集画面へ戻る。

まだ保存されていません

以下の内容で上書きしてよろしいですか？

項目	変更後のデータ
日付	2025-12-15
項目名	ゲーム課金
値段	60000 円
購入目的	ガチャ
備考欄	200連UR3体 (*°▽°*)

修正する

確定して保存

修正したい場合は「修正する」ボタンを押す
→ 1 つ前の編集画面に戻る

図 9: 編集内容確認画面

5. 「確定して保存」ボタンを押すと、データが上書き保存され、詳細画面へ戻る（図10）。一覧表示にも変更が反映されている（図11）。

まだ保存されていません

以下の内容で上書きしてよろしいですか？

項目	変更後のデータ
日付	2025-12-15
項目名	ゲーム課金
値段	60000 円
購入目的	ガチャ
備考欄	200連UR3体 (*°▽°*)

修正する

確定して保存

ゲーム課金

項目	データ
値段	60000 円
日付	2025-12-15
購入目的	ガチャ
備考欄	200連UR3体 (*°▽°*)

[編集する](#)

[一覧に戻る](#)

[この項目を削除する](#)

図 10: 編集完了後の画面

お小遣い帳

[新規項目を追加する](#)

日付	項目名	値段
2025-12-10	カレー	250 円
2025-12-12	おいお茶	120 円
2025-12-15	ゲーム課金	30000 円
合計金額		30370 円

[トップページへ](#)

お小遣い帳

[新規項目を追加する](#)

日付	項目名	値段
2025-12-10	カレー	250 円
2025-12-12	おいお茶	120 円
2025-12-15	ゲーム課金	60000 円
合計金額		60370 円

[トップページへ](#)

図 11: 編集前と後の一覧表示画面

2.6 データの追加

新しいデータを登録する手順は以下の通りである。

1. 一覧表示画面の上方にある「新規項目を追加する」リンクをクリックする（図 12）。

お小遣い帳

[新規項目を追加する](#)

日付	項目名	値段
2025-12-10	カレー	250 円
2025-12-12	おいお茶	120 円
2025-12-15	ゲーム課金	30000 円
合計金額		30370 円

[トップページへ](#)

図 12: 新規登録画面への行き方

2. 新規登録画面が表示される (図 13). 日付, 項目名, 値段 (半角数字), 購入目的, 備考欄を入力する.

新規項目の追加

項目	データ
日付	年 / 月 / 日 📅 ※日付を選択
項目名	
値段	0 円 ※半角数字で入力
購入目的	
備考欄	

確認画面へ

[キャンセル \(一覧に戻る\)](#)

図 13: 新規登録画面

3. 入力が完了したら「確認画面へ」ボタンを押す (図 14).

新規項目の追加

項目	データ
日付	2025/12/25 📅 ※日付を選択
項目名	おもちゃ
値段	1980 円 ※半角数字で入力
購入目的	プレゼント
備考欄	喜んでくれた

確認画面へ

[キャンセル \(一覧に戻る\)](#)

図 14: 新規登録入力例と「確認画面へ」ボタン

4. 新規登録内容の確認画面が表示される (図 15). 内容に間違いがなければ「追加する」ボタンを押す. 修正が必要な場合は「修正する」ボタンを押して入力画面へ戻る.

まだ保存されていません

以下の内容で登録してよろしいですか？

項目	データ
日付	2025-12-25
項目名	おもちゃ
値段	1980 円
購入目的	プレゼント
備考欄	喜んでくれた

修正する

追加する

修正したい場合は「修正する」ボタンを押す
→ 1 つ前の編集画面に戻る

図 15: 新規登録内容確認画面

5. 「追加する」を押すとデータが保存され, 一覧表示画面に追加されたデータが表示される (図 16).

まだ保存されていません

以下の内容で登録してよろしいですか？

項目	データ
日付	2025-12-25
項目名	おもちゃ
値段	1980 円
購入目的	プレゼント
備考欄	喜んでくれた

修正する

追加する

お小遣い帳

[新規項目を追加する](#)

日付	項目名	値段
2025-12-10	カレー	250 円
2025-12-12	おーいお茶	120 円
2025-12-15	ゲーム課金	30000 円
2025-12-25	おもちゃ	1980 円
合計金額		32350 円

[トップページへ](#)

図 16: 新規登録完了後の画面

2.7 データの削除

不要になったデータを削除する手順は以下の通りである。

1. 削除したいデータの詳細画面を開き，赤字で表示された「削除する」リンクをクリックする（図 17）。

お小遣い帳

[新規項目を追加する](#)

日付	項目名	値段
2025-12-10	カレー	250 円
2025-12-12	おいお茶	120 円
2025-12-15	ゲーム課金	30000 円
2025-12-25	おもちゃ	1980 円
合計金額		32350 円

[トップページへ](#)

おもちゃ

項目	データ
値段	1980 円
日付	2025-12-25
購入目的	プレゼント
備考欄	喜んでくれた

[編集する](#)
[一覧に戻る](#)

[この項目を削除する](#)

図 17: 削除画面への行き方

2. 削除確認画面が表示される（図 18）。この操作は取り消せないため、内容をよく確認する。削除しない場合は「キャンセル（詳細に戻る）」リンクをクリックする。

本当に削除しますか？

以下のデータが完全に消去されます。元に戻すことはできません。

項目	データ
日付	2025-12-25
項目名	おもちゃ
値段	1980 円
購入目的	プレゼント
備考欄	喜んでくれた

削除する

[キャンセル（詳細に戻る）](#)

図 18: 削除画面

3. 本当に削除してよければ「削除する」ボタンを押す（図 19）。

本当に削除しますか？

以下のデータが完全に消去されます。元に戻すことはできません。

項目	データ
日付	2025-12-25
項目名	おもちゃ
値段	1980 円
購入目的	プレゼント
備考欄	喜んでくれた

削除する

[キャンセル（詳細に戻る）](#)

削除したい場合のみ「削除する」ボタンを押す
これ以降データを復元できない

図 19: 削除の実行

4. データが削除され、一覧画面へ戻る（図 20）。一覧表示画面からは削除した項目が無くなっている。

お小遣い帳

[新規項目を追加する](#)

日付	項目名	値段
2025-12-10	カレー	250 円
2025-12-12	おーいお茶	120 円
2025-12-15	ゲーム課金	30000 円
2025-12-25	おもちゃ	1980 円
合計金額		32350 円

[トップページへ](#)

➡

お小遣い帳

[新規項目を追加する](#)

日付	項目名	値段
2025-12-10	カレー	250 円
2025-12-12	おーいお茶	120 円
2025-12-15	ゲーム課金	30000 円
合計金額		30370 円

[トップページへ](#)

※一度削除したデータは復元不可

図 20: 削除後の一覧表示画面

3 管理者向けドキュメント

3.1 インストール方法

3.1.1 システム本体の準備

本システムのソースコードは GitHub にて公開されている。GitHub の URL (https://github.com/Kuro-aletta/webpro_06) からリポジトリをダウンロード、またはターミナル上でソースコード 1 を実行し、任意のディレクトリに配置する。

ソースコード 1: システム本体のクローン

```
1 $ git clone https://github.com/Kuro-aletta/webpro_06.git
```

3.1.2 ライブラリのインストール

本システムは Node.js 上で動作し、Express および EJS パッケージを使用する。ターミナルを開き、システムを配置したディレクトリへ移動した後、ソースコード 2 のコマンドを実行して必要なライブラリをインストールする。

ソースコード 2: ライブラリのインストール

```
1 $ npm install
```

これにより、`package.json` に記載されている、本システムの実行に必要なライブラリ (`express`, `ejs`) がまとめてインストールされる。

3.2 起動方法

ディレクトリ内に `app_report.js` および `node_modules` フォルダが存在することを確認する。確認後、ソースコード 3 のコマンドを実行してサーバを起動する。

ソースコード 3: システムの起動

```
1 $ node app_report.js
```

「Example app listening on port 8080!」と表示されれば起動成功である。Web ブラウザで以下の URL にアクセスして動作を確認する。

- トップページ: `http://localhost:8080/`
- 成績管理システム: `http://localhost:8080/result`
- 教員情報システム: `http://localhost:8080/teacher`
- お小遣い帳システム: `http://localhost:8080/wallet`

3.3 起動できない場合

3.3.1 ライブラリが見つからない場合

起動時に `Error: Cannot find module 'express'` と表示される場合は、ライブラリのインストールが行われていない。 `npm install` コマンド (ソースコード 2) を再度実行することで解決する。

3.3.2 ポート番号が既に使用されている場合

起動時に `Error: listen EADDRINUSE: address already in use :::8080` と表示される場合は、既に別のプロセスが 8080 番ポートを使用している。これは多くの場合、前回のサーバ起動時に正しく終了できず、バックグラウンドでプロセスが動作し続けていることに起因する。この場合、以下の手順でプロセスを特定し、停止させる必要がある。

1. ポート 8080 を使用しているプロセスの ID (PID) を特定する (ソースコード 4).

ソースコード 4: PID の確認

```
1 $ lsof -i :8080
```

2. 表示された PID (数値) を指定してプロセスを終了 (kill) する (ソースコード 5).

ソースコード 5: プロセスの終了

```
1 $ kill <PID>
```

3. 上記コマンド (ソースコード 5) でも終了しない (強制終了が必要な) 場合は, -9 を付けて再度実行する (ソースコード 6).

ソースコード 6: プロセスの強制終了

```
1 $ kill -9 <PID>
```

なお, `app_report.js` 内のポート番号設定 (`app.listen(8080, ...)` の部分) を別の番号 (8081 など) に書き換えることでも対応可能であるが, アクセスする URL が変更となり利用者への周知が必要となるため, この方法は推奨しない.

3.4 終了方法

サーバを停止させる場合は, 起動しているターミナル上で `Ctrl + C` キーを入力する. なお, `Ctrl + Z` はプロセスを一時停止させる操作であり, 完全に終了しない. この状態でターミナルを閉じるとポートが占有されたままとなり, 次回の起動時にエラー (EADDRINUSE) の原因となるため注意が必要である.

3.5 分かっている不具合・制限事項

本システムはデータをサーバのメモリ上に保存しているため, サーバを再起動または終了すると, 登録したデータはすべてリセットされる仕様である. なお, 現在確認されている動作上の不具合はない.

4 開発者向けドキュメント

4.1 概要

本システムは、サーバサイド JavaScript 実行環境である Node.js および Web フレームワーク Express を用いて構築された Web アプリケーションである。アーキテクチャとして MVC (Model-View-Controller) モデルに近い構成を採用しており、`views` ディレクトリ内の EJS テンプレート (View) と、`app_report.js` によるルーティングおよびロジック処理 (Controller) により動作する。なお、本システムではデータベースを使用せず、サーバメモリ上の変数 (配列) としてデータを管理する (Model 相当) 仕様としている。

4.2 開発環境

本システムの開発および動作環境は以下の通りである。

- 実行環境: Node.js (v24.3.0)
- 言語: JavaScript
- フレームワーク: Express
- テンプレートエンジン: EJS

4.3 ディレクトリ構造

GitHub (https://github.com/Kuro-aletta/webpro_06) にて公開されている本システムのファイル構成を図 21 に示す。なお、本システムに関与しないファイルは省略されている。実行の起点となるプログラムである `app_report.js` をルートに置き、静的リソース (CSS など) は `public` ディレクトリ、画面のひな型となる EJS ファイルは `views` ディレクトリに配置して管理している。

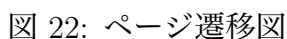
webpro_06/	
├─ app_report.js	サーバサイドプログラム
├─ package.json	依存ライブラリ定義
├─ node_modules/	インストールされたライブラリ
├─ public/	静的ファイル用ディレクトリ
│ ├─ style.css	共通スタイルシート
│ └─ top.html	トップページ
└─ views/	EJS ファイル用ディレクトリ
├─ result_list.ejs	成績: 一覧
├─ result_detail.ejs	成績: 詳細
├─ result_add.ejs	成績: 追加入力
├─ result_add_confirm.ejs	成績: 追加確認
├─ result_edit.ejs	成績: 編集入力
├─ result_edit_confirm.ejs	成績: 編集確認
├─ result_delete_confirm.ejs	成績: 削除確認
├─ teacher_list.ejs	教員: 一覧
├─ teacher_detail.ejs	教員: 詳細
├─ teacher_add.ejs	教員: 追加入力
├─ teacher_add_confirm.ejs	教員: 追加確認
├─ teacher_edit.ejs	教員: 編集入力
├─ teacher_edit_confirm.ejs	教員: 編集確認
├─ teacher_delete_confirm.ejs	教員: 削除確認
├─ wallet_list.ejs	財布: 一覧
├─ wallet_detail.ejs	財布: 詳細
├─ wallet_add.ejs	財布: 追加入力
├─ wallet_add_confirm.ejs	財布: 追加確認
├─ wallet_edit.ejs	財布: 編集入力
├─ wallet_edit_confirm.ejs	財布: 編集確認
└─ wallet_delete_confirm.ejs	財布: 削除確認

図 21: ディレクトリ構造

4.4 システム構造と URL 設計

本システムは、app_report.js を実行し、views/ ディレクトリ内の EJS ファイルをもとに HTML を生成して表示する構成をとっている。データの永続化は行わず、サーバメモリ上の変数（配列）としてデータを保持する仕様とした。

各システムにおけるページ遷移図を図 22 に、HTTP メソッド、リソース名 (URL) の対応を表 1 に示す。



画面・機能	HTTP メソッド	URL パターン
トップページ	GET	/
一覧表示	GET	/リソース名
詳細表示	GET	/リソース名/detail/:number
新規登録（入力）	GET	/リソース名/add
新規登録（確認）	POST	/リソース名/add/confirm
新規登録（修正）	POST	/リソース名/add/return
新規登録（実行）	POST	/リソース名/create
編集（入力）	GET	/リソース名/edit/:number
編集（確認）	POST	/リソース名/edit/confirm/:number
編集（修正）	POST	/リソース名/edit/return/:number
編集（実行）	POST	/リソース名/update/:number
削除（確認）	GET	/リソース名/delete/confirm/:number
削除（実行）	POST	/リソース名/delete/:number
エラー（404）	ALL	-

各システムのリソース名は以下の通りである。

- 成績管理システム: result
- 教員情報システム: teacher
- お小遣い帳システム: wallet

以下に、本システムの画面遷移やデータの受け渡しに関する詳細な仕組みを述べる。

4.4.1 ページ遷移の仕組み

各画面間の遷移は、主に以下の2つの方法で行われる。

- 参照 (GET) : 一覧画面の各項目やメニューのリンク (<a>タグ) をクリックすることで遷移する。
- 操作 (POST) : 登録・編集・削除の確認画面や実行処理へは、フォーム (<form>タグ) の送信ボタンをクリックすることで遷移する。

4.4.2 データの受け渡し (リレー方式)

新規登録や編集の際、入力画面から確認画面を経て実行処理へと進む過程で、一度サーバに送信したデータを維持する必要がある。本システムでは、HTTP の直前の状態を保持しない特性を補うため、確認画面のフォーム内に type="hidden" 属性を持つ input タグを配置し、次画面へ再度データを送信して受け渡す「リレー方式」を採用した。

4.4.3 処理実行後の画面遷移

データの追加・更新・削除が完了した後の遷移先は、ユーザビリティを考慮して以下のように設計した。

- 追加・削除後: 作業結果全体を確認しやすくするため、「一覧表示画面」へリダイレクトする。
- 編集後: 変更内容を即座に確認できるようにするため、そのデータの「詳細表示画面」へリダイレクトする。

4.5 データ構造

各システムは、オブジェクトの配列としてデータを管理している。それぞれのデータ構造を以下に示す。

4.5.1 成績管理システム (result)

履修科目の成績情報を管理する。

表 2: 成績データの構造

項目名	変数名	説明
年度	year	開講年度 (数値)
セメスタ	semester	開講学期 (数値)
科目名	name	講義の名称
担当教員	teacher	教員名
副担当	subteacher	副担当教員名 (任意)
単位数	credit	科目の単位数 (数値)
成績	grade	S, A, B, C, 合 など

4.5.2 教員情報システム (teacher)

教員の所属や担当科目などの情報を管理する。

表 3: 教員データの構造

項目名	変数名	説明
学科	department	所属学科名
氏名	name	教員名
職位	job	教授, 准教授など
研究室	laboratory	研究室名
担当科目	subject	主な担当科目

4.5.3 お小遣い帳システム (wallet)

金銭の出納記録を管理する。合計金額の計算に使用するため, price は必ず数値型として扱う。

表 4: お小遣い帳データの構造

項目名	変数名	説明
日付	date	利用日 (YYYY-MM-DD 形式)
項目名	item	購入品目などの名称
値段	price	金額 (数値)
購入目的	purpose	利用の目的
備考欄	memo	メモ

4.6 各システムの機能詳細

4.6.1 成績管理システム

履修データの登録・編集・削除を行う。一覧画面では、科目名 (name) をクリックすることで詳細画面へ遷移する。教員名が未入力の場合は「(科目名を設定してください)」と表示することで、詳細画面へ遷移できなくなる状況を防いでいる。表示制御として、成績 (grade) が未入力の場合には「-」を、科目名が未入力の場合には注意書きとして「(科目名を設定してください)」を表示するようにしている。詳細画面においては、副担当教員 (subteacher) が登録されていない (空文字の) 場合、テーブルの行自体を表示しないようにしている。また、成績 (grade) が未入力の場合には「未評価」を表示するようにしている。

4.6.2 教員情報システム

教員データの管理を行う。一覧画面では、教員名 (name) をクリックすることで詳細画面へ遷移する。教員名が未入力の場合は「(教員名を設定してください)」と表示することで、詳細画面へ遷移できなくなる状況を防いでいる。詳細画面では、担当科目 (subject) が登録されていない場合に「なし」と表示する処理を加えている。

4.6.3 お小遣い帳システム

支出データの管理を行う。一覧画面では、項目名 (item) をクリックすることで詳細画面へ遷移する。項目名が未入力の場合は警告として「(項目名を設定してください)」を表示することで、詳細画面へ遷移できなくなる状況を防いでいる。本システム特有の機能として、一覧表示を行う際、サーバ側で登録済みデータの値段 (price) をすべて加算し、合計金額を算出する処理を実装している。算出された合計値は一覧表の最下部に表示される。

4.7 講義範囲外の要素の実装

本システムの開発にあたり、ユーザビリティの向上やデータの整合性を保つため、いくつかの工夫を取り入れた。

4.7.1 EJS 内での条件分岐 (if 文)

`result_list.ejs` などにおいて、データが空文字の場合の表示を制御するために EJS 内で if 文を使用した。例えば、成績が空の場合は「-」を表示し、入力済みの場合はそのままを表示することで、視認性を向上させている。

ソースコード 7: EJS での条件分岐の例

```
1 <% if (row.grade == "") { %>
2   -
3 <% } else { %>
4   <%= row.grade %>
5 <% } %>
```

4.7.2 CSS によるデザインの管理

本システムでは、画面の見た目（デザイン）を HTML ファイル（.ejs）から分離し、一括管理するために CSS（Cascading Style Sheets）を導入した。public フォルダに `style.css` ファイルを作成し、各 EJS ファイルから `<link>` タグを用いて読み込んでいる。これにより、HTML 内に `border="3"` などの属性を記述することなく、全ページで統一されたデザインを適用している。今回実装した主な CSS の機能を表 5 に示す。

表 5: 定義した CSS クラスとスタイル

セレクタ/クラス名	対象の要素	機能・効果
<code>table</code>	<code><table></code>	外枠を 3px の実線にし、枠線を結合する
<code>th</code>	<code><th></code>	背景色を灰色にし、文字を拡大、下線を太くする
<code>td</code>	<code><td></code>	内側の枠線を 1px にし、余白（padding）を設ける
<code>.text-red</code>	クラス指定	文字色を赤にする（削除確認メッセージ等）
<code>.text-blue</code>	クラス指定	文字色を青にする（登録確認メッセージ等）
<code>.delete</code>	クラス指定	背景色を薄い赤にする（削除ボタン用）
<code>input[type="submit"]</code>	送信ボタン	余白を設け、カーソルを指の形にする
<code>.total-border</code>	クラス指定	上線を 3px の実線にする（合計金額欄用）

EJS ファイル側では、ソースコード 8 のように `class` 属性を指定して、CSS で定義したスタイルを適用している。

ソースコード 8: CSS クラスの適用例

```
1 <h2 class="text-redまだ保存されていません"></h2>
2
3 <td class="total-border合計金額"></td>
```

4.7.3 入力補助のための small タグ

新規登録や編集画面において、入力項目の下に `<small>` タグを用いて注釈（例：「※半角数字で入力」など）を追加した。これにより、ユーザが期待されるデータ形式を直感的に理解できるようにした。

4.7.4 数値計算と入力値の検証

お小遣い帳システムにおいて、フォームから送信されたデータは通常「文字列」としてサーバに受け渡される。そのままでは正しい加算処理（合計金額の算出）ができないため、ソースコード 9 のように、データの登録時および更新時に JavaScript の `parseInt()` 関数を使用して明示的に整数型へ変換して保存する処理を実装した。また、金額が未入力（空文字）の場合に計算結果が `NaN`（非数値）になるのを防ぐため、`isNaN()` 関数を用いた入力値のチェックを行い、数値でない場合は 0 として扱うよう堅牢性を高めた。

ソースコード 9: 数値変換と検証の処理

```
1 // フォームからの入力を数値に変換
2 let priceVal = parseInt(req.body.price);
3
4 // 数値でない場合（空欄など）はにする 0
5 if (isNaN(priceVal)) {
6   priceVal = 0;
7 }
```

5 おわりに

本レポートでは、成績管理、教員情報、お小遣い帳という 3 つの Web アプリケーションの設計と実装について報告した。統一された操作体系を持つシステムを複数構築することで、Web 開発における共通のパターンを深く理解することができた。

本課題を通じて、Web アプリケーションにおける基本的な CRUD 操作の実装方法を習得した。特に、データの更新や削除といったデータの変更を伴う操作には POST メソッドを使用し、単なる参照には GET メソッドを使用するという Web の標準的な設計ルール的重要性を理解した。また、確認画面を挟む実装においては、一度サーバにデータを送信した後、`hidden` 属性を用いて再度データを次の画面へ受け渡すリレー方式を採用した。これにより、通信のたびに接続が切れる（直前の状態を保持しない）HTTP の特性にお

いて、画面間でデータを引き継ぐ方法を学んだ。さらに、CSS を用いて見た目のデザインと HTML の構造を分けて管理することや、データの型を意識した入力値のチェックなど、実用的な Web システム開発に必要な周辺技術についても理解を深めることができた。

今回はデータの保存をメモリ上の変数で行ったため、サーバ停止とともにデータがリセットされる仕様であるが、今後はデータベースを用いた永続化など、より実用的なシステムへの発展が課題である。